

fNIRS Manual

Table of Contents

Data acquisition

1. Orientation of fNIRS Room	1
2. Putting on the Headcap (10-20 System)	5
3. Optode Templates	6
4. Setting Up a Measurement	7
5. Digitizing Optodes	9
6. Adding Events	10
7. Initiating the Measurement	12

Data analysis

8. Downloading Oxysoft data	13
9. Converting .oxy files to .nirs	14
10. Converting .nirs to .snirf	15
11. Accessing Jupyter with MNE	17
12. Practice Notebooks	19
13. Notebook 1 Key Sections	20
a. Loading fNIRS data	20
b. Setting up Digitization	21
c. Experiment Events	23
d. Preprocessing fNIRS Data	24

Data Acquisition

Orientation of fNIRS Room

- The room used to conduct fNIRS measurements is 405H (first room on your left when you walk in), located in the Human Neuroscience Laboratory (Mcguinn 405)
- When you walk into the room, you will see a yellow briefcase directly in front of you
- **This briefcase contains all things fNIRS**
 - We recommend plugging in and starting up the fNIRS laptop as soon as you arrive
- The desktop computer on your *left* when you walk in is mainly used for experimental display. PsychoPy is already installed on the computer and can be found in the top left corner of the desktop

Briefcase Components

Briefcase	Keeps the system safe and makes it easy to carry around	
Brite 24	Completely portable NIRS system with 27 channels (Clips easily attach it to the head cap) Gently twist transmitters and receivers into optode holders (you will hear a small click when transmitters/receivers are in place) (this can be done before being put on the participant's head)	

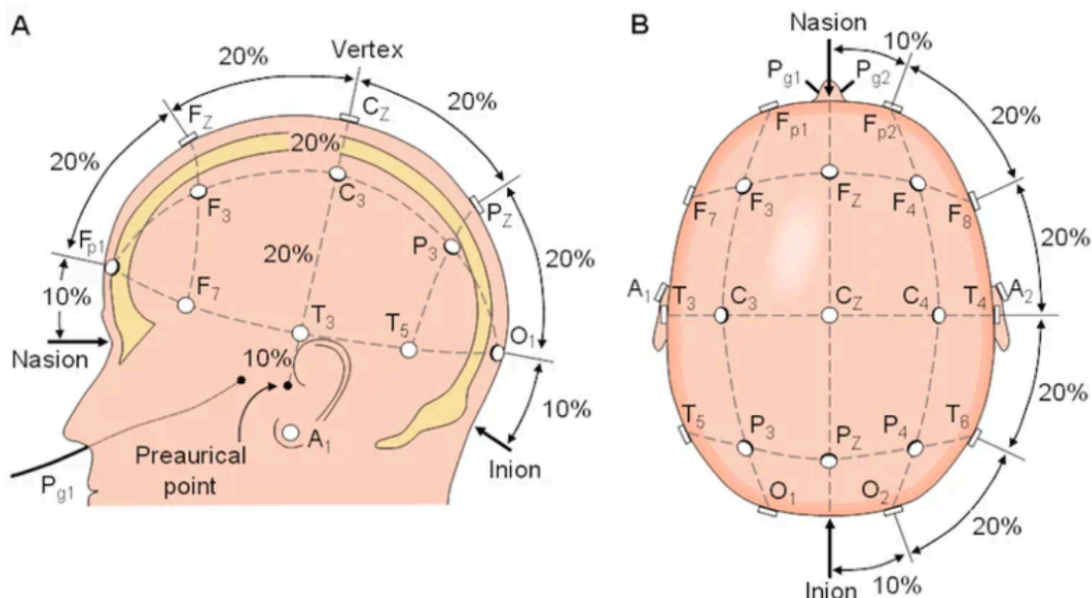
	The Brite device can be rested on the participant's shoulder	
Bluetooth Dongle	Connects the Brite to the laptop via Bluetooth	
Bluetooth Antenna	Used to increase the distance between the Brite and the laptop when using Bluetooth dongle Gets connected directly to the dongle Not necessary when the participant is stationary in close proximity to the laptop	
License Key (ROCKY)	USB that can be used to access NIRS software (Oxysoft) This MUST be plugged into the laptop port labeled "Rocky" before Oxysoft can even be opened. It must remain plugged in throughout the entirety of data acquisition and analysis	

USB	Contains software, manuals, data examples to practice with, scientific articles, and some general NIRS info	
Optode holders	Used to place the receivers and the transmitters of the Brite in the headcap (Yellow for transmitters) (Blue for receivers) From the info provided in CortiVision fNIRS Tut..., stretch cap and put the optode holders in the desired holes	
Distance Guards	Makes measurements easier and control optode spacing, but are not necessary if you are using the 10-20 EEG system to place electrodes. Put around optode holders BEFORE optodes are attached	
Brite Charger (2)	Charges the brite device We recommend keeping the Brite device plugged in throughout the duration of the experiment—it loses battery quickly.	

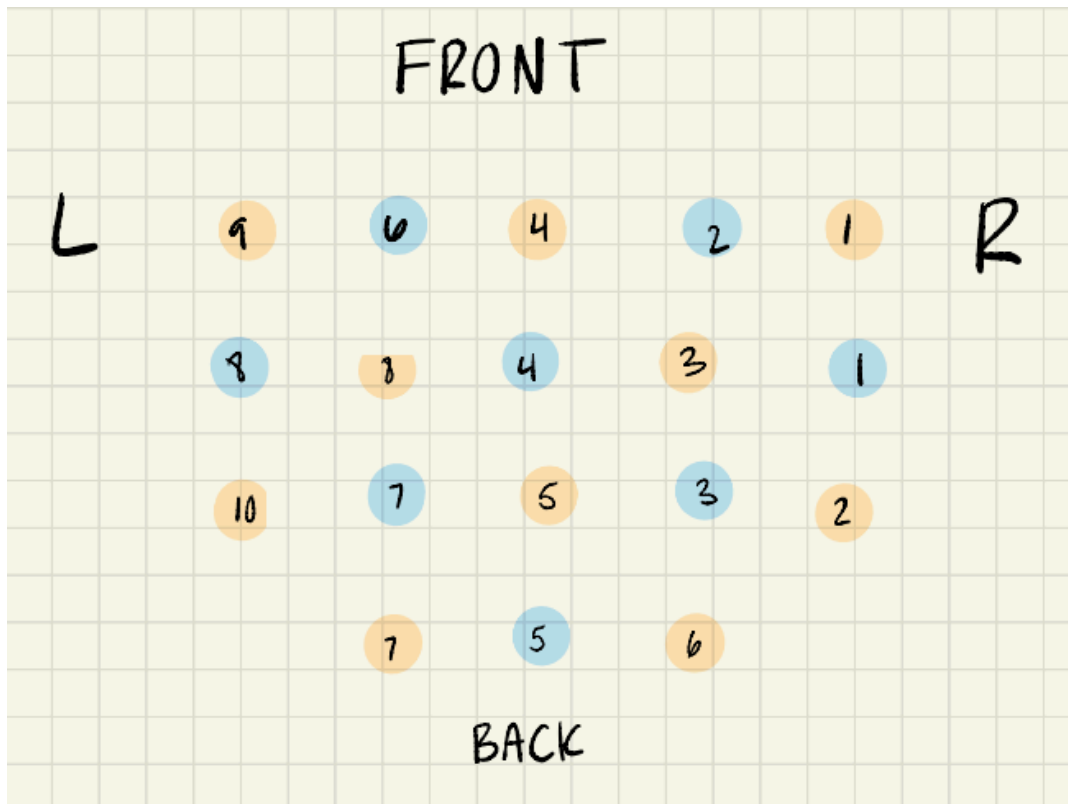
USB Cable	Connects Brite device directly to laptop	
Laptop	Has free installed software that can be used for data acquisition and analysis Recommended to present stimuli on the desktop to the left on the room's entrance so NIRS acquisition software doesn't run out of memory	
Headcaps	Allow for standardized spacing of electrodes. We have three standard sizes (S, M, L) We recommend using size large for adolescent-adult participants.	

Putting on the Headcap (10-20 System)

- Based on advice from Artinis representatives and a vast array of fNIRS literature, we recommend using 10-20 EEG capping procedures
- From PubMed: **fNIRS setup on the participant.** We placed the anterior ridge of the fNIRS cap on the participant's forehead such that the middle anteriormost point of the cap ridge laid directly over the midline frontal polar 10/20 location. Next, we pulled the lateral straps of the cap down such that the cap wrapped snugly around the participant's head. It was important to secure the cap under the participant's chin with Velcro® straps, making sure that the original placement of the cap ridge remained in place over the eyebrows and was situated evenly across the left and right hemispheres of the head. Before attaching the fNIRS optodes to the cap, cord management/support devices were set up. Next, we examined the condition of the hair underneath the cap; fNIRS optodes should be in direct contact with the scalp. If any hair impeded the fNIRS optodes, we used a blunt syringe and water-soluble gel to move the hair at each optode location. In our study, two people set up the optodes to reduce the setup time. After each optode was securely fastened to the participant's head, we conducted a signal calibration and quality scan. In case of poor signal quality, we corrected the placement of the optodes in the affected channels by gently “wiggling” them in place or carefully removing them and repeating the steps outlined above.
- See the below videos for visual examples of proper capping procedures:
 - <https://www.youtube.com/watch?v=d0ERFWZkNZO> (“CortiVision fNIRS Tutorial - Putting on a cap” by Cortivision)
 - <https://www.youtube.com/watch?v=dAB2QEKifzI> (“Cortivision fNIRS tutorial - Hair management for better signal quality” by Cortivision)



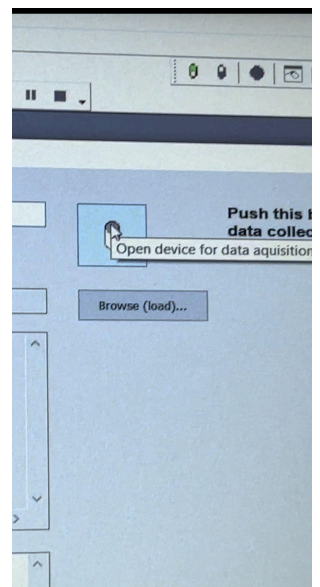
Optode Templates



- Numbers highlighted in orange represent transmitters, numbers highlighted in blue represent receptors
- The distance between transmitter/receiver pairings should be about 30mm.
- One transmitter may output to multiple receivers, and one receiver may receive input from multiple transmitters
- This template is NOT meant to cover the entire cap, but rather to be shifted depending on an experiment's regions of interest (i.e. the occipital lobe for visual experiments)
- This template represents one possible configuration, but many variations can be found online

Setting Up a Measurement

- The Brite optodes can be found in the briefcase
 - **If possible, leave it plugged in (charger found in the yellow briefcase) throughout the duration of your measurement, as it tends to lose battery fairly quickly**
- To turn the Brite on, either plug it in OR hold down the Brite button for 2-3 seconds
 - It will begin to glow green when it is on
- You should also see a blue flashing light towards the bottom of the Brite, indicating that Bluetooth connection is online and searching
- If this light stops flashing, restart the Brite by holding the Brite button down for a few seconds to shut it off, then holding it down again to turn it back on
- To gain access to Oxysoft, the **red Rocky flash drive** must be plugged into the right side of the computer
 - The rocky can be found in the yellow briefcase, and the USB it is meant to be plugged into is labeled on the laptop
- Once you've plugged in the Rocky, you will be able to access Oxysoft from the top left corner of the desktop
- Open a new project by selecting File > New Project File
 - Upon opening the software, you may see a prior measurement still up on the home page
 - **Be sure to save this project file to ensure that the prior researchers don't lose their data**
 - Once you select the New Project File button, a popup will appear asking if you want to save the current project
 - **Always say yes just to be safe!**
- Initiate a new measurement by selecting Measurement > Create Offline Measurement
- Then, from the Select setting dropdown menu on the ensuing popup, select Brite23
 - Then press OK.
- Then, name your measurement on the following screen (make sure to give it a very descriptive name to avoid confusion)
 - On this same screen, press the "Open device for data acquisition" button
- After pressing this button, a popup screen will appear for you to add a device
 - On this screen, press the "Add..." button in the top right corner
 - **Ensure that the blue Bluetooth light on the Brite optode system is flashing before you do this**



■ **If it is not, restart the Brite following the instructions above (holding down the Brite center button)**

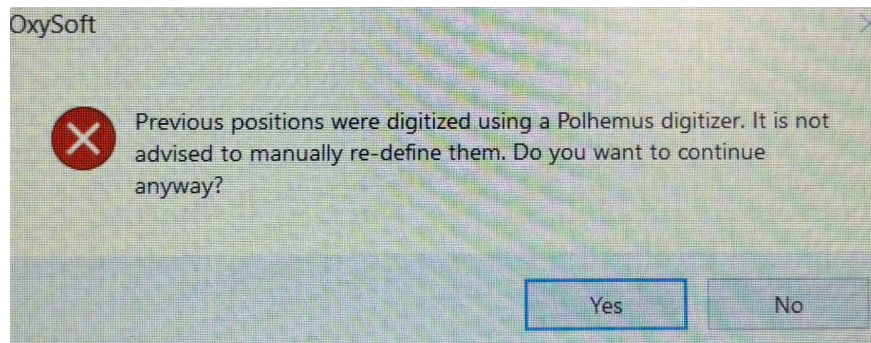
- This will prompt a second pop-up screen - under the drop-down for “Device type to add” select Brite23
 - The device will pop up as Type: Brite, ID: 24115
 - Once this occurs, press OK
 - Then, press next
- On the following screen, you will be given the option to input an optode-template and a DPF value:

Sub	DPF	d (mm)
Brite23	6	35 ▼

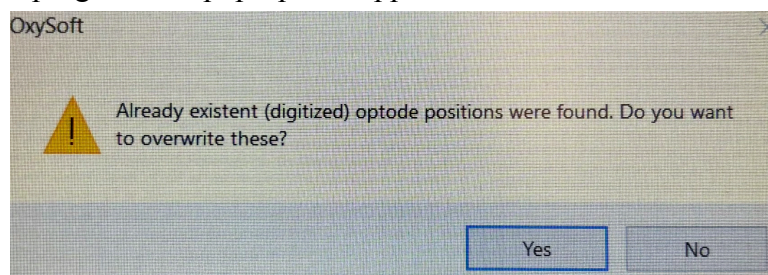
- From the “Optode-template” dropdown, select the “Brite24 27 channel,” as this is the most effective template for our system
- The DPF value is calculated by age, and Oxysoft will output a value for you if you select the “Calculate DPF from Age (BRAIN)” button
 - Paste this value into the DPF column of the table
- d should be kept at 35 mm, as this is the standardized distance between optodes
- After inputting all of the above data, press Next
- The next screen will show light wavelength and optode/transmitter pairings
 - **You do not need to do anything on this screen**
 - Essentially, this screen is just indicating that every transmitter emits a slightly different wavelength of light such that the optodes know which transmitters they’re receiving from
 - Press Next to continue
- On the following screen, select 50 Hz from the Sample dropdown, as higher sampling frequencies are shown to produce better fNIRS data
 - Nothing else needs to be done on this page
 - Press Next
- The pop-ups will now disappear, and you will find yourself on the main OxySoft page

Digitizing Optodes

- Once the measurement has been set up, you must digitize the optodes before starting data acquisition
- Select “Measurement” from the top left, and press “Digitize Optodes”
- Two options will appear: “Polhemus digitizer” and “Manual digitization”
 - *The Human Neuro Lab does not currently have a Polhemus digitizer, so select Manual digitization to proceed*
- The following pop-up will appear:



- This is a pre-programmed warning that will not negatively affect data acquisition
 - Select “Yes” to continue
- The following screen will show a 2D topographic view of the optodes on the left, with a 3D view to the right
 - Edit optode placements on the 2D screen, and the 3D screen will automatically update
 - Drag and drop receptors (R_) and transmitters (T_) to their respective locations using the 10-20 system and the key locations highlighted on the 2D topographic map
 - Once finished, select OK
- Another pre-programmed pop-up will appear:



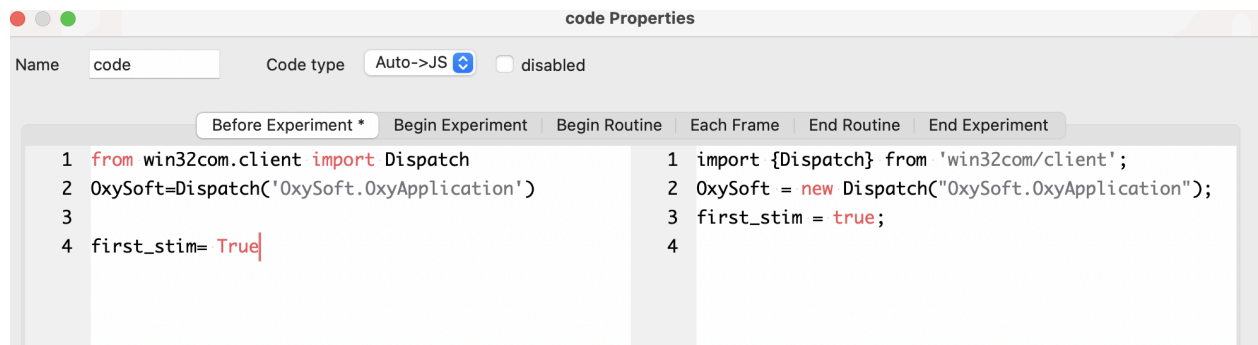
- Select Yes to overwrite these
- Oxysoft will ask if you want to save the project after successful digitization
 - This is not necessary, as you will save your project after data acquisition
 - However, if you want to be extra safe (it never hurts), select yes

Adding Events

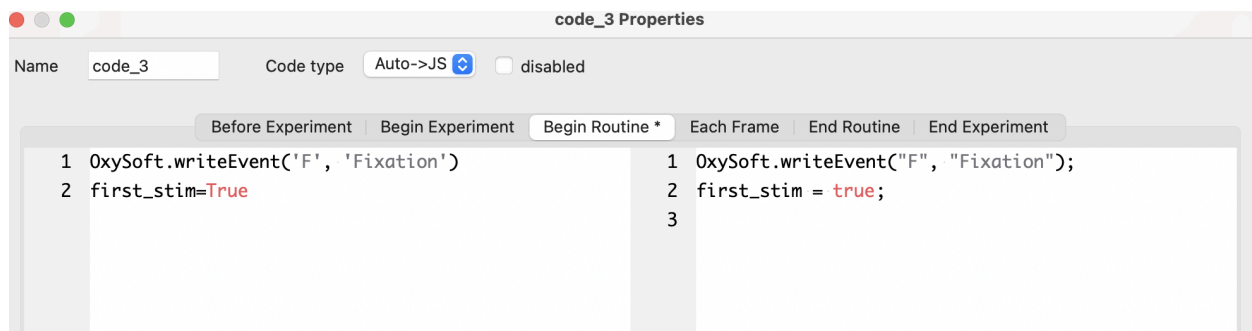
- For most experiments, you will use Psychopy to present the stimuli
- Though you can run Psychopy on a separate computer from Oxysoft, we found it easiest to send event signals if both softwares are running on the same computer (in this case, the Dell laptop)
- Therefore, Psychopy has been installed on the laptop to run experiments
- In order to send signals between Psychopy and Oxysoft, one must add a series of code to their Psychopy experiment
- To begin, select the first routine in your experiment
- Under the “Components” section on the right side of Psychopy, select “Custom” and then “Code”
- Copy the following code:


```
from win32com.client import Dispatch
OxySoft=Dispatch('OxySoft.OxyApplication')
```

 - This should be pasted in the “Before Experiment” section of the code properties, and the code type should be set to “Auto->JS”



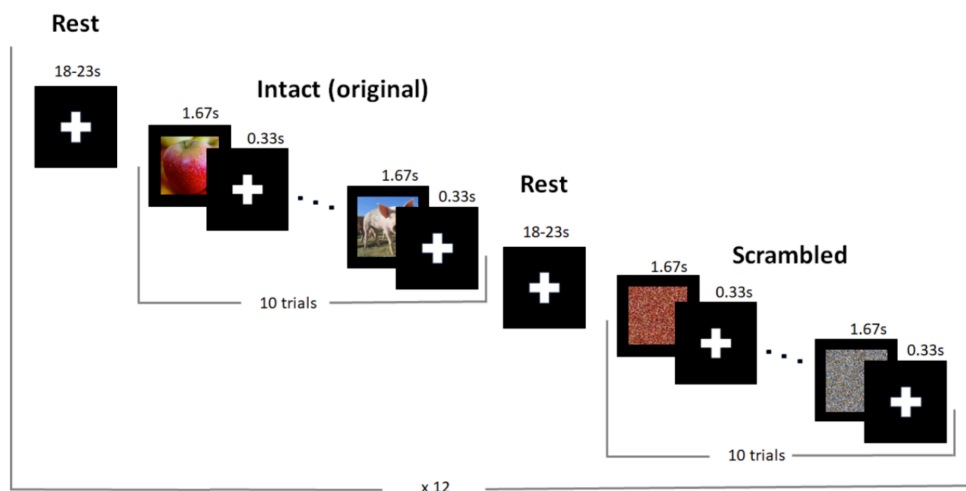
- NOTE: first_stim=True is specific to the study being run and does not have to be included
- Now, under any routine in which you want an event sent to Oxysoft, place code formatted like the following:



- May sure to write this under the “Begin Routine” section of the code properties with the code type set to “Auto -> JS”
- NOTE: Our routine here was a fixation, explaining the choice of “(‘F’, ‘Fixation’)” - if your routine was something else, say, an object being displayed, you could replace this with “(‘O’, ‘Object’)” as the letter is what will be displayed on your Oxysoft recording
- NOTE: first_stim=True is specific to the study being run and does not have to be included
- Once you have added code for every routine you want marked as an event in Oxysoft, you are finished
 - Events will be sent to Oxysoft once the Psychopy experiment has started AND the Oxysoft recording begins
 - NOTE: The events will not appear in the Oxysoft recording in real time, only after the recording on Oxysoft has been ended
- The link below will bring you to the practice study we did with the corresponding Psychopy file
 - It includes all the study materials including stimuli, psychopy file, and fNIRS data
 - Below is a depiction of the paradigm

Object Viewing Practice Study

3. Presentation of the task



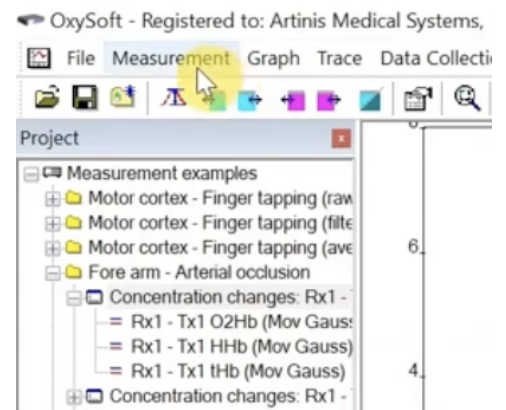
Initiating the Measurement

- On the left of the screen, you will see a DAQ Status section
- Under this section, you will see “Device Status” and, underneath it, “Idle (click here to start)”
 - Click this
- A pop-up will emerge asking if you are ready to begin emitting light
 - There are two **critical** things you must ensure before selecting yes:
 - (1) Ensure that your paradigm (whether it be Psychopy, Matlab, etc.) is up and ready
 - You want to ensure that you sync the experiment’s start with the start of recording
 - NOTE: This is only really necessary if you're paradigm does not have events connected to Oxysoft, as the events will be time linked
 - (2) You’ve limited all extraneous sources of light that may interfere with your measurement
 - Be sure to close the door, turn off overhead lights/any unnecessary electronics, and ensure that there are no unnecessary gaps between optodes (i.e. unfilled holes in the headcap) before beginning your measurement
- Once both tasks above are completed, select “Yes” to begin the measurement
 - The device will indicate that it’s recording under the DAQ status section
- The DAQ menu will also create a popup that displays the strength of your optode connections
 - If highlighted in red, adjust the corresponding optode(s) to ensure complete contact with scalp
 - Do not proceed until all optodes are no longer red
 - It is necessary to have good connection or otherwise data will not meet the processing threshold
- Begin your experiment directly after
- When you are finished recording, select the stop measurement button under the same DAQ section

Data Analysis

Downloading Oxysoft Data

- Highlight (click) the measurement in your project window that you want to export, then select measurement in the toolbar
- Select “Export Measurement” from the dropdown menu
- You will then be given a list of export types
 - For our purpose (analysis in the MNE toolbox), select .oxy4 file, then click next
- You will then be prompted to select a file destination
 - Click browse and choose the folder you want to export to
 - You will also assign a file name at this time
 - Click “Save” and then “Next” once finished
- When you’ve reached the final page of the pop-up, click “Start Export” to export the data

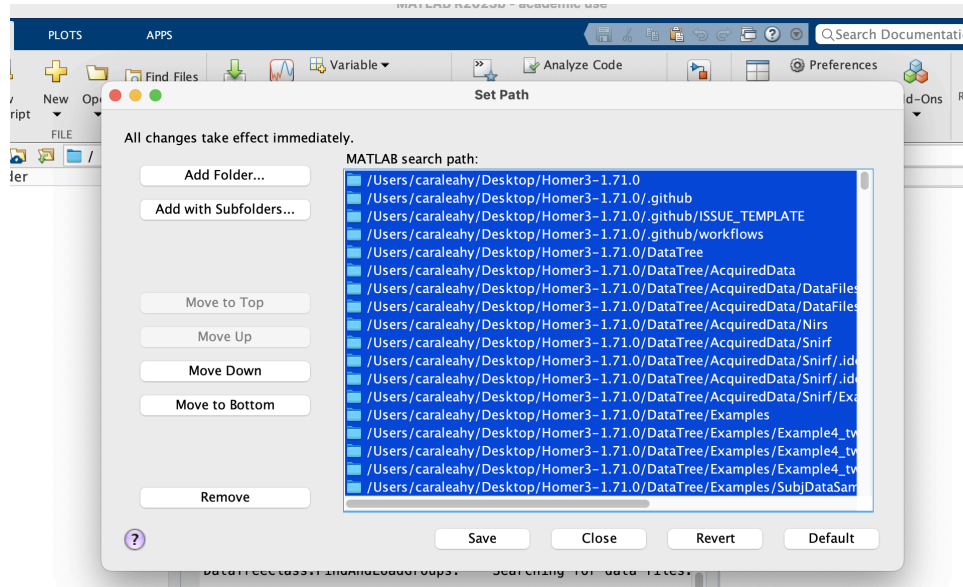


Converting .oxy files to .nirs

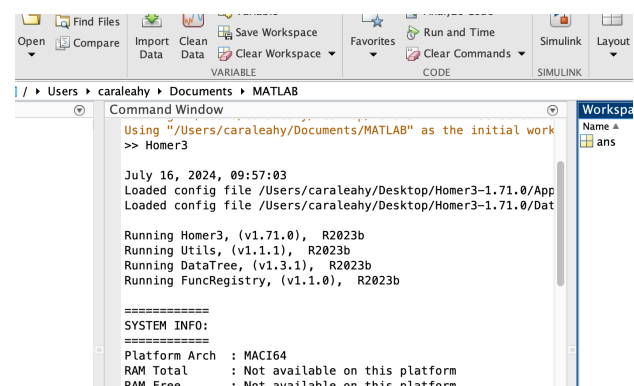
- Oxysoft exports its data as .oxy files, but the MNE toolbox will only read .snirf files
- In order to convert a .oxy file to a .nirs file, one must start by converting the .oxy file to a .nirs file
- To do this, you will need to use the Dell laptop that has a script downloaded by Artinis entitled “oxysoft2matlab”
 - This script is used in Matlab to convert the data
- Once you have downloaded the data you want to export, open Matlab
- Add to path the folder in which the oxysoft2matlab script is stored
 - NOTE: The oxysoft2matlab script and the .oxy file you want to convert should be in the same folder
- In the command window, type oxysoft2matlab
- Matlab will then ask for a series of inputs regarding the code in order to run
- After running this command, a .nirs file will be created in the same folder in which the .oxy file is in
- From here, we recommended adding the .nirs file to your google drive for easier access
 - To do this, open outlook on the laptop and log into Google Drive, then import the file

Converting .nirs files to .snirf

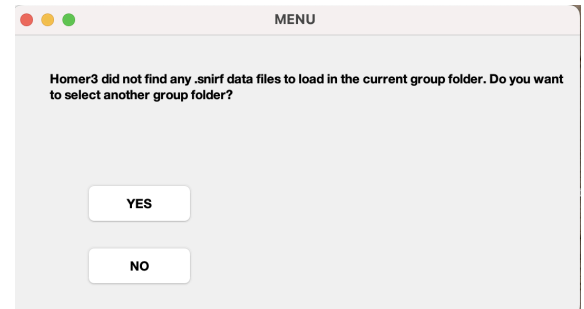
- To use MNE, you will now need to convert your .nirs file to .snirf
 - NOTE: Download your .nirs file from your Google Drive before beginning
- To do this, you will utilize Homer, another fNIRS software
 - NOTE: There may be another way to convert to .snirf, but we used this way as we were already familiar with Homer and had it downloaded
- Homer can be installed here: <https://openfnirs.org/software/homer/homer3/>
 - NOTE: Homer is a Matlab application, so you will need Matlab to run Homer
 - If you do not have Matlab downloaded, you can do so by visiting this site <https://www.bc.edu/bc-web/offices/its/services/research-services/software.html#tab-matlab>
- Once Homer has been installed, open Matlab
- Within Matlab, select “Set Path” in the right corner
- A popup window will open - select “Add with Subfolders” on the left side
- From here, select your Homer download folder (this will likely be labeled Homer3-)



- Click open once highlighted
- Now, you should see that Matlab has added the files from your Homer3 folder to its search path
 - It should look similar to the image above
- Select save and then close the popup window



- Then, in the command window, write “Homer3”
- If installed correctly, Homer should begin running
- You should then receive a popup similar to the following, depending on where you have saved your .nirs file to
 - At this point, you can either add your .nirs file to the Homer folder, or you can select “Yes” and choose the folder in which you have saved your .nirs file to
- Once Homer has found your .nirs file(s), it will ask you if you want to create a .snirf file
- Select yes, and a .snirf file will be created within the same folder as your .nirs file
- You may then quit the application



Accessing Jupyter with MNE

- MNE is a python package used to analyze fNIRS data

Pybrain: M/EEG analysis with MNE Python

- Above is a useful video for accessing Jupyter Lab with MNE, specifically the time point 6:50 - 25:49 (“Installation and Conda setup”)
- To use MNE, install Anaconda, which is linked here: <https://www.anaconda.com/download>
- Follow application download steps until anaconda has been installed (you should see an image like the one on the right)
- With anaconda installed, open your computer terminal (which looks like this  for Apple users)
- Within your terminal, run the following code:

`conda config --set channel_priority flexible`

NOTE: To run the code (highlighted in gray), simply type what is written above and then press the return key

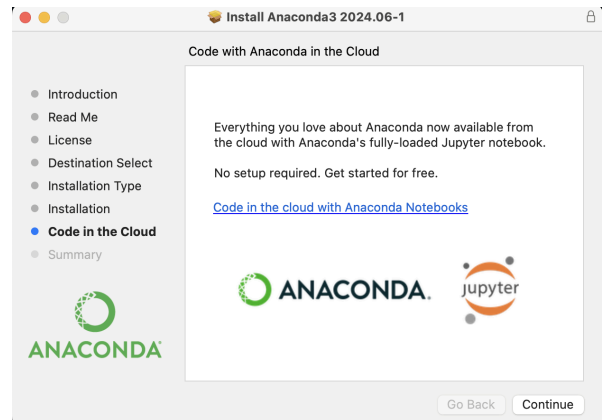
- Now, run this line of code

`conda config --set solver libmamba`

- Now, run this line of code

`conda install --channel=conda-forge --name=base conda-libmamba-solver`

NOTE: You will be given the option to “Proceed” while running this code (see image below), simply write enter “y” to continue



```
python_abi          conda-forge/osx-arm64::python_abi-3.12-4_cp312

The following packages will be UPDATED:

ca-certificates      pkgs/main::ca-certificates-2024.7.2-h~ --> conda-forge::ca-
certificates-2024.7.4-hf0a4a13_0
openssl              pkgs/main::openssl-3.0.14-h80987f9_0 --> conda-forge::ope
nssl-3.3.1-hfb2fe0b_1
zlib                 pkgs/main::zlib-1.2.13-h18a0788_1 --> conda-forge::zli
b-1.2.13-hfb2fe0b_6

The following packages will be SUPERSEDED by a higher-priority channel:

certifi              pkgs/main/osx-arm64::certifi-2024.7.4~ --> conda-forge/noar
ch::certifi-2024.7.4-pyhd8ed1ab_0
conda                pkgs/main::conda-24.5.0-py312hca03da5~ --> conda-forge::con
da-24.5.0-py312h81bd7bf_0
conda-libmamba-solv~ pkgs/main::conda-libmamba-solver-24.1~ --> conda-forge::con
da-libmamba-solver-24.1.0-pyhd8ed1ab_0
python               pkgs/main::python-3.12.4-h99e199e_1 --> conda-forge::pyt
hon-3.12.2-hdf0ec26_0_cpython

Proceed ([y]/n)? y
```

- Now, run this line of code

```
conda create --solver=libmamba --override-channels --channel=conda-forge --name=mne mne
```

NOTE: This will also have the “Proceed” option like the one above, so follow the same steps (write “y”)

NOTE: This step will take a few minutes to complete

- Now, run this line of code

```
conda activate mne
```

- Now, run this line of code

```
mne sys_info
```

NOTE: This step is just to ensure you successfully installed MNE - if you have, the package settings should be displayed after running this line

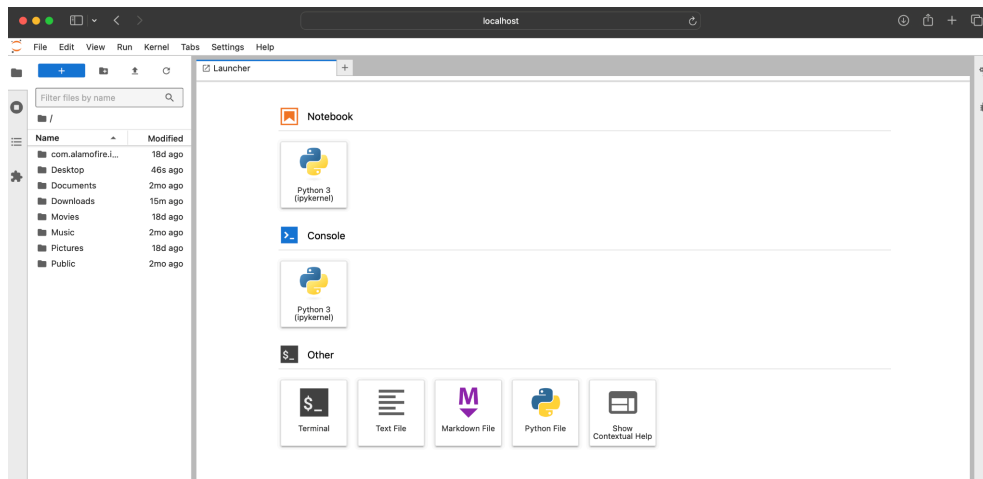
- Now, run this line of code

```
conda install -y jupyterlab
```

- Now, run this line of code

```
jupyter lab
```

- If all is successful, your terminal should open Jupyter in a new tab
 - It should look like the image below



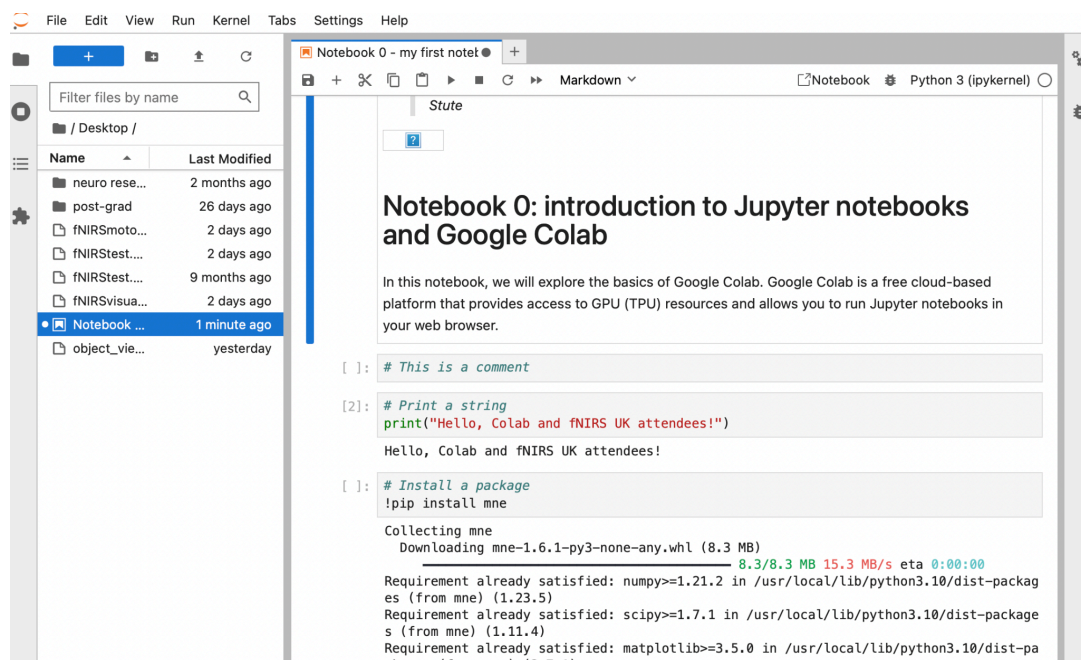
- From this point on, whenever you want to open Jupyter to perform data analysis with MNE, you will do the following
 - Open terminal and run the following line


```
conda activate mne
```
 - Followed by


```
jupyter lab
```
- NOTE: When closing out of terminal it will ask “Do you want to terminate running processes in this window” - Click “Terminate” to close the window

Practice Notebooks

- To get familiar with MNE, we followed along with a workshop, linked here: <https://brain-data-uon.gitlab.io/events/fnirsuk2023-workshop.html>
- On the site, click “this presentation” (highlighted in blue) to access their presentation
- On the second slide of the presentation, they have linked their google drive which contains their practice notebooks
 - Download each notebook you want to complete, as well as the folder titled “Example Data” in order to run the code in Jupyter
 - NOTE: Their data is a .snirf file, so no need to do any of the conversion steps
- With the notebooks and example data downloaded, open Jupyter
- On the left hand side, travel to where you have stored the notebooks on your computer
- Open Notebook 0, 1, or 2 and get started!
 - Notebook 0 - Serves as an intro to Jupyter Notebooks
 - Notebook 1 - Serves as an intro to MNE subject level analysis
 - Notebook 2 - Serves as an intro to MNE group level analysis
 - NOTE: The notebook has been opened correctly if it looks similar to the image below
- NOTE: To run a line of code in a jupyter notebook, either press the play button on the top left or hold the shift and return keys simultaneously
 - You will know a line has run if something displays below the line and/or a number appears in the bracket box [] to the left of the line (the number corresponds to how many times the code has been run)
- NOTE: Anything with “colab” mentioned can be disregarded, as that is specific to Google Colab - which you are not using
- NOTE: When finished, make sure to save by clicking the button in the top left corner of the notebook before exiting the page

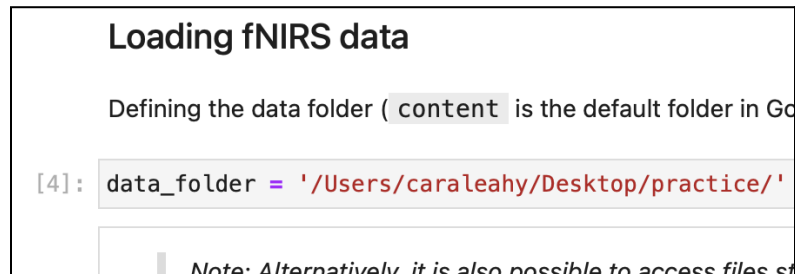


Notebook 1 Key Sections

- There are a few key sections in Notebook 1 that you may experience difficulty running

Loading fNIRS data

- In this section you will define your data folder (or in other words, where you have stored your .snirf file(s))
- To define your data folder, you will set the variable “data_folder” to equal where your data folder is on your computer
 - To do this, see image below
 - The set up of this code is as followed
`'/Users/(your Mac login username)/(where the folder is stored such as desktop, downloads, etc)/(the folder name)'`
 - NOTE: you may have to add a section to this code depending on where it is stored in your computer
 - In any case, just make sure your code is formatted so that it follows the pipeline you would use to access the data folder yourself
 - NOTE: It is imperative that you include all apostrophes ‘ and slashes / as seen in the image in order for the code to run
 - You will know it has successfully ran if no error appears below your code
- NOTE: This process is for Mac users and may differ on Windows
- To check if you have set the variable to the correct folder, run the segment of code beneath this line (see image below)
 - This code will check for files in your data folder that end with .snirf
 - If you have a .snirf file in your data folder, and you have correctly assigned the variable “data_folder” to your data folder, then a print out similar to the one seen in the image should appear below your code

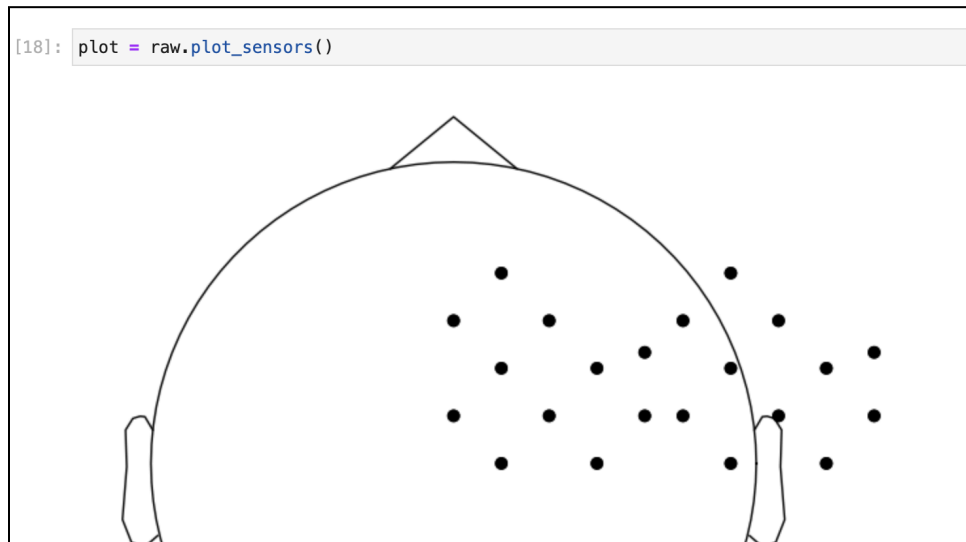


```
[14]: import glob
snirf_file = glob.glob(data_folder + '*.snirf')[0]
snirf_file

[14]: '/Users/caraleahy/Desktop/practice/20230707_KD01_25hz.snirf'
```

Setting up Digitization

- In order to plot your optodes for a 2d or 3d rendering, you will need to tell python where the optodes were located with respect to the participants head
- NOTE: As you may remember from data acquisition, you will be using the coordinates that you set from your manual digitization
- To get started, first plot the optode sensors on a default template by running the following code
 - This should display an image like the one seen below



- Now, to match up where the optodes are on the screen to where they were on the participant, you will need to create an elc file
 - NOTE: If you do not want or know how to manually create an elc file, we recommend downloading the elc file create from the workshop we mentioned earlier and editing the file from there (linked here: [Example Data](#))
- Download or create the elc file and add it to the data folder on your computer
 - The downloaded elc file should look like the image to the right
 - NOTE: To open the elc file, right click on the file and select “Open with” and then “TextEdit”
- There are two “sections” to this file, labeled “Positions” and “Labels”
 - These two sections correspond to each other, with the “Positions” section giving the three dimensional position of the optode labeled below in the “Labels” section

```

# ASA optode file
ReferenceLabel avg
UnitPosition mm
NumberPositions= 21
Positions
1.0 83.6 -48.6
80.3 -19.7 -43.9
-82.6 -20.1 -43.1
50.20 -89.28 48.17
30.96 -112.73 8.43
73.12 -70.89 -1.33
58.66 -93.47 -12.13
-50.20 -89.28 48.17
-30.96 -112.73 8.43
-73.12 -70.89 -1.33
-58.66 -93.47 -12.13
26.85 -104.70 41.38
75.00 -64.84 5.60
57.10 -93.33 21.32
30.32 -111.53 -24.31
74.81 -62.20 -27.51
-26.85 -104.70 41.38
-75.00 -64.84 5.60
-57.10 -93.33 21.32
-30.32 -111.53 -24.31
-74.81 -62.20 -27.51
Labels
Nz
RPA
LPA
D1
D2
D3
D4
D5
D6
D7
D8
S1
S2
S3
S4
S5
S6
S7
S8
S9
S10

```

- Under “Labels” at the bottom of the file, you will see Nz, RPA, LPA
- The Nz, RPA, and LPA coordinates are standard and should NOT be changed
- You will also see “D_” which represents detectors (or otherwise known as receivers) 1-8 and “S_” which represents signals (or otherwise known as transmitters) 1-10
- To get the three dimensional coordinates for your transmitters and receivers, you will follow steps 1-3 (outlined below) in Oxysoft (the document with which these steps came is linked here: https://openfnirs.org/wp-content/uploads/wpforo/default_attachments/1626728103-oxysoft_export_to_matlab_toolboxes.pdf)

You can retrieve the projected positions of the channels from Oxysoft by following these instructions:

1. Right click on the 3D plot in Oxysoft and select “Show MNI coordinates”.
2. Activate the “Output to MS Excel file” option on the lower right, go to the “MS Excel” tab and select the output file.
3. Go back to the “Calculation” tab and click on “Calculate”. Then, the Excel file is created.

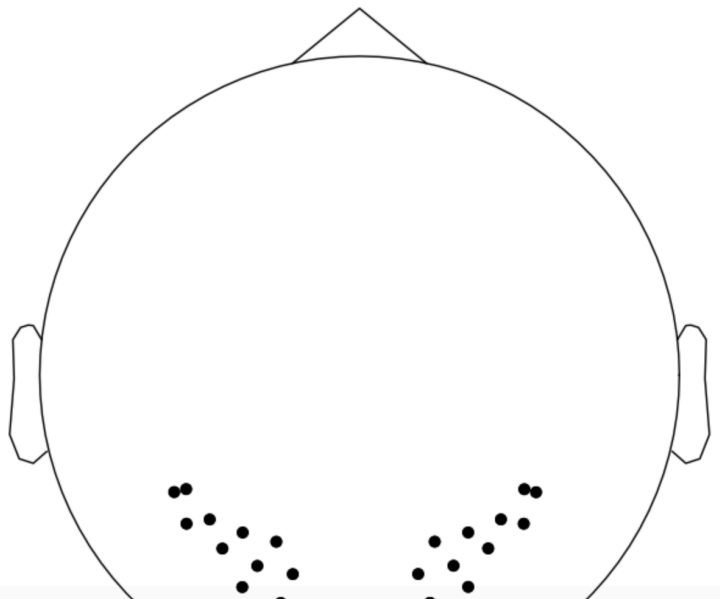
t +31 481 350 980
w www.artinis.com
e askforinfo@artinis.com

Einsteinweg 17
6662 PW Elst
The Netherlands

artinis
medical systems

- Once you have obtained the coordinates, add them to your elc file in the order that corresponds to the “Labels” section of the file
- Save the elc file, and then return to your Jupyter notebook
- Run the line of code as seen below, making sure that your elc file (written in red below) is named the same in both this line of code and on your computer
- If everything was done correctly, beneath your code should appear a 2d rendering of a set up similar to the one that was seen on the participants head

```
[22]: montage = mne.channels.read_custom_montage(data_folder + 'digitisation.elc')
raw.set_montage(montage)
plot = raw.plot_sensors()
```



Preprocessing fNIRS Data

- In this section and in other places of the notebook there will be sections of code for short separation channels
- The lab does not currently own these channels, so disregard those segments in your analysis
- Within preprocessing, you will remove channels that are deemed as “bad”, or in other words, have a low SCI (Scalp Coupling Index -> a measurement of the cardiac signal or “noise” that interferes with the evaluation of the hemodynamic response)
- To do this, you will first run the following code to determine your channels SCI values

```
[25]: raw_od = mne.preprocessing.nirs.optical_density(raw) # intensity to relative op
raw_od = mne.preprocessing.nirs.temporal_derivative_distribution_repair(raw_od)
```

Computing the scalp coupling index (SCI) to access channel quality (cf. [Pollonini et al., 2014](#)):

```
[26]: sci = mne.preprocessing.nirs.scalp_coupling_index(raw_od)
sci
```

```
[26]: array([0.99960033, 0.99960033, 0.99983214, 0.99983214, 0.99966217,
            0.99966217, 0.99939512, 0.99939512, 0.99935459, 0.99935459,
            0.99857526, 0.99857526, 0.99833804, 0.99833804, 0.99982892,
            0.99982892, 0.99848328, 0.99848328, 0.99944206, 0.99944206,
            0.99988111, 0.99988111, 0.9996801 , 0.9996801 , 0.99972373,
            0.99972373, 0.99983581, 0.99983581, 0.99975014, 0.99975014,
            0.99621207, 0.99621207, 0.99520644, 0.99520644, 0.99843841,
            0.99843841, 0.99925824, 0.99925824, 0.99864922, 0.99864922,
            0.99988173, 0.99988173, 0.99767885, 0.99767885])
```

- Channels with an SCI of under 0.7 are considered bad and are therefore removed from the analysis
 - To do this, run the following code segment

```
[27]: raw_od.info["bads"] = list(np.array(raw_od.ch_names)[sci < 0.7])
raw_od.info["bads"]
```

```
[27]: []
```

Deleting bad channels:

```
[28]: raw_od.pick('fnirs', exclude='bads')
```

- NOTE: 0.7 is the recommended value, but can be adjusted if too many of your channels are under this value - adjust on a case by case basis